

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE ESTUDIOS SUPERIORES ARAGÓN
INGENIERÍA EN COMPUTACIÓN

Documento Integral del Proyecto: Sistema de Incidencias

Presenta:

Ariel Hernández Velázquez

Asignatura:

Vinculación Empresarial

Fecha de entrega:

29 de mayo de 2026

Índice general

Resumen Ejecutivo	6
1. Introducción	7
2. Planteamiento del Problema	8
3. Objetivos	9
3.1. Objetivo General	9
3.2. Objetivos Específicos	9
4. Marco Teórico	10
4.1. Sistemas de Información	10
4.2. Desarrollo Web Full-Stack	10
4.3. Bases de Datos Relacionales	10
4.4. Gestión de Incidencias	11
4.5. Contenedores y Docker	11
4.6. Control de Versiones con Git	11
5. Alcance del Sistema	12
5.1. Qué hará el sistema	12
5.2. Qué NO hará el sistema	12
6. Requerimientos del Sistema	14
6.1. Requerimientos Funcionales	14
6.2. Requerimientos No Funcionales	15

6.3. Reglas de Negocio	16
7. Casos de Uso	17
8. Historias de Usuario	19
9. Tecnologías Seleccionadas	21
9.1. Backend	21
9.2. Frontend	21
9.3. Base de Datos	22
9.4. Contenedores y Despliegue	22
9.5. Control de Versiones	22
10.Arquitectura del Sistema	23
10.1. Capa de Presentación	23
10.2. Capa de Lógica de Negocio	23
10.3. Capa de Datos	24
10.4. Seguridad	24
11.Estructura del Proyecto y Distribución	25
11.1. Rol de los módulos	26
12.Uso de Contenedores y Despliegue	27
13.Modelo de Base de Datos	28
13.1. Tablas y relaciones	28
13.2. Índices	29
13.3. Integridad referencial	29
14.Estrategia de Control de Versiones	30
15.Riesgos del Proyecto	31
16.Conclusiones	32

16.1. Beneficios del sistema	32
16.2. Aprendizajes del desarrollo	32
Matriz de Trazabilidad	35

Índice de figuras

7.1. Diagrama de casos de uso del Sistema de Incidencias	17
10.1. Arquitectura general cliente-servidor	23
13.1. Modelo entidad-relación simplificado	28

Índice de cuadros

6.1. Requerimientos Funcionales	15
6.2. Requerimientos No Funcionales	15
6.3. Reglas de Negocio	16
7.1. Descripción de casos de uso	18
8.1. Historias de usuario	20
13.1. Descripción de tablas	29
15.1. Matriz de riesgos	31
16.1. Matriz de trazabilidad objetivos vs requerimientos	35

Resumen Ejecutivo

El sistema de gestión de incidencias de limpieza para la FES Aragón surge ante la necesidad de digitalizar y optimizar los reportes de problemas de aseo en aulas y laboratorios. Actualmente, el flujo manual basado en formatos físicos y comunicación verbal genera retrasos, pérdida de información y falta de trazabilidad. La solución desarrollada es una aplicación web full-stack que permite a usuarios reportar incidencias, a revisores (personal de limpieza) actualizar el estado de las tareas y a administradores gestionar catálogos y analizar estadísticas.

El sistema emplea tecnologías modernas: Node.js con Express en el backend, MariaDB como gestor de base de datos relacional, y un frontend nativo con HTML5, CSS3 y JavaScript. La autenticación se realiza mediante JSON Web Tokens (JWT) para garantizar seguridad y sesiones sin estado. El proyecto está completamente contenedorizado con Docker, lo que facilita su despliegue en cualquier entorno de producción. Entre las funcionalidades destacadas se incluyen paginación, historial de cambios, subida de imágenes y exportación a CSV.

El beneficio principal es la reducción del tiempo de respuesta en la atención de incidencias, la transparencia en el flujo de trabajo y la disponibilidad de indicadores para la toma de decisiones. El alcance del sistema abarca tres perfiles de usuario, gestión completa de edificios y salones, y generación de reportes estadísticos. Se espera que su implementación mejore significativamente la calidad del servicio de limpieza en la institución.

Capítulo 1

Introducción

En el ámbito educativo, la infraestructura física y su mantenimiento son factores críticos para el desarrollo académico. La FES Aragón cuenta con múltiples edificios y salones que requieren atención constante en materia de limpieza. El método tradicional para reportar incidencias se basa en formularios impresos o mensajes electrónicos no estructurados, lo que provoca ineficiencias, duplicidad de reportes y dificultad para realizar seguimiento.

La digitalización de este proceso mediante una aplicación web ofrece numerosas ventajas: centralización de la información, automatización de flujos de trabajo, trazabilidad de cada reporte y generación de métricas. El presente documento describe el desarrollo completo del Sistema de Incidencias, desde su análisis y diseño hasta su implementación técnica. Se abordan los requerimientos funcionales y no funcionales, la arquitectura cliente-servidor, el modelo de datos, y las estrategias de control de versiones y despliegue con Docker.

El sistema está orientado a tres actores: usuarios (comunidad universitaria que reporta problemas), revisores (personal de limpieza que atiende las incidencias) y administradores (coordinadores que gestionan catálogos y visualizan estadísticas). Con esta herramienta se busca mejorar la comunicación entre la comunidad y el área de mantenimiento, reduciendo tiempos de respuesta y proporcionando indicadores confiables.

Capítulo 2

Planteamiento del Problema

La gestión de incidencias de limpieza en la FES Aragón enfrenta las siguientes problemáticas:

- **Proceso manual y descentralizado:** Los reportes se realizan mediante hojas de papel o correos electrónicos a distintas áreas, sin un registro unificado.
- **Falta de trazabilidad:** No se lleva un historial de cambios ni se sabe quién atendió cada incidencia, ni en qué momento se resolvió.
- **Duplicidad de reportes:** Un mismo problema puede ser reportado varias veces por diferentes personas, generando esfuerzos redundantes.
- **Dificultad para priorizar:** Al no contar con métricas, es imposible identificar los edificios o salones con mayor frecuencia de incidencias.
- **Retraso en la atención:** La comunicación verbal o por medios informales provoca que las incidencias queden olvidadas o se atiendan tarde.
- **Pérdida de información:** Los formatos físicos se extravían o deterioran, y los correos quedan dispersos en múltiples bandejas.

Estas inefficiencias impactan directamente en la calidad del servicio de limpieza, generando insatisfacción en la comunidad universitaria y riesgos sanitarios. Un sistema digital que automatice el ciclo de vida de las incidencias (creación, asignación, cambio de estado, resolución) permitirá eliminar los problemas mencionados.

Capítulo 3

Objetivos

3.1. Objetivo General

Desarrollar un sistema web de gestión de incidencias de limpieza para la FES Aragón que permita reportar, dar seguimiento y analizar problemas de aseo en aulas y laboratorios, mejorando la eficiencia y trazabilidad del proceso.

3.2. Objetivos Específicos

1. Diseñar e implementar una base de datos relacional que almacene usuarios, roles, edificios, salones, incidencias e historial de cambios, garantizando integridad referencial.
2. Construir un backend con Node.js y Express que exponga una API REST segura mediante autenticación JWT, con operaciones CRUD para incidencias, edificios, salones y usuarios.
3. Desarrollar un frontend responsivo con HTML, CSS y JavaScript que ofrezca interfaces diferenciadas para usuarios, revisores y administradores, incluyendo paginación, filtros y subida de imágenes.
4. Implementar un historial de cambios que registre cada modificación de estado junto con el usuario responsable y un comentario opcional.
5. Contenerizar la aplicación utilizando Docker y Docker Compose, permitiendo su despliegue rápido en cualquier entorno de producción o desarrollo.

Capítulo 4

Marco Teórico

4.1. Sistemas de Información

Un sistema de información es un conjunto de componentes interrelacionados que recolectan, procesan, almacenan y distribuyen información para apoyar la toma de decisiones y el control en una organización. En este proyecto, el sistema de gestión de incidencias se clasifica como un sistema de soporte a operaciones (TPS - Transaction Processing System) debido a que registra transacciones diarias (reportes, cambios de estado) y produce informes.

4.2. Desarrollo Web Full-Stack

La arquitectura full-stack comprende tres capas: frontend (interfaz de usuario), backend (lógica de negocio) y base de datos. El frontend se implementa con tecnologías nativas (HTML5, CSS3, JavaScript) para garantizar ligereza y compatibilidad. El backend utiliza Node.js, un entorno de ejecución basado en el motor V8 de Chrome, que permite construir aplicaciones escalables y de alto rendimiento mediante un modelo asíncrono orientado a eventos. Express.js es el framework minimalista que facilita la definición de rutas, middlewares y manejo de peticiones HTTP.

4.3. Bases de Datos Relacionales

MariaDB (derivado de MySQL) es el sistema gestor de bases de datos elegido. Su modelo relacional organiza los datos en tablas con filas y columnas, garantizando integridad referencial mediante claves primarias y foráneas. El lenguaje de consultas SQL permite realizar operaciones de inserción, actualización, eliminación y selección. Para este proyecto se normalizaron las tablas hasta la tercera forma normal (3NF) para evitar redundancias y anomalías.

4.4. Gestión de Incidencias

Una incidencia es un evento anormal relacionado con la limpieza que requiere ser reportado, asignado, atendido y resuelto. Su ciclo de vida típico incluye los estados: Pendiente, En Proceso y Resuelto. La trazabilidad se logra mediante un historial de cambios que registra la transición de estados junto con la fecha y el usuario responsable.

4.5. Contenedores y Docker

Docker es una plataforma de virtualización a nivel de sistema operativo que permite empaquetar aplicaciones y sus dependencias en contenedores ligeros. La orquestación con Docker Compose facilita la definición y ejecución de múltiples contenedores (base de datos y aplicación) mediante un archivo YAML. Esto asegura la reproducibilidad del entorno en desarrollo, pruebas y producción.

4.6. Control de Versiones con Git

Git es un sistema de control de versiones distribuido que permite gestionar el código fuente, registrar cambios y colaborar entre desarrolladores. El flujo de trabajo GitFlow emplea ramas principales (main y develop) y ramas auxiliares (features, releases, hotfixes) para organizar el desarrollo. Las políticas de commits semánticos y pull requests mejoran la calidad del código.

Capítulo 5

Alcance del Sistema

5.1. Qué hará el sistema

- Permitir a los usuarios autenticados reportar incidencias seleccionando edificio, salón y proporcionando una descripción.
- Permitir a los revisores (personal de limpieza) listar incidencias filtradas por estado, cambiar el estado (Pendiente → En Proceso → Resuelto) agregando un comentario, y subir fotografías como evidencia.
- Permitir a los administradores gestionar edificios (CRUD), salones (CRUD), listar usuarios, visualizar estadísticas de incidencias por edificio y estado, exportar reportes a CSV y ver el historial completo de cada incidencia.
- Almacenar automáticamente un historial de cambios cada vez que se modifica el estado de una incidencia.
- Proporcionar paginación en todas las listas para manejar grandes volúmenes de datos.
- Ejecutarse en contenedores Docker para facilitar el despliegue.

5.2. Qué NO hará el sistema

- No se integrará con servicios de autenticación externos como Google OAuth (aunque está preparado para hacerlo, simula un login con correo y rol).
- No enviará notificaciones automáticas por correo electrónico o mensajería instantánea.
- No gestionará horarios de limpieza ni asignación automática de personal.
- No permitirá adjuntar múltiples imágenes por incidencia (solo una).
- No incluirá un panel de control en tiempo real con WebSockets.

- No tendrá una aplicación móvil nativa (solo versión web responsiva).

Capítulo 6

Requerimientos del Sistema

6.1. Requerimientos Funcionales

ID	Descripción	Prioridad
RF-01	El sistema debe permitir a cualquier usuario registrarse automáticamente al iniciar sesión con correo, nombre y rol.	Alta
RF-02	El sistema debe autenticar a los usuarios mediante JWT y expirar el token después de 8 horas.	Alta
RF-03	El sistema debe mostrar interfaces diferenciadas según el rol: Usuario, Revisor, Administrador.	Alta
RF-04	El sistema debe permitir a los usuarios reportar incidencias seleccionando edificio, salón y escribiendo una descripción.	Alta
RF-05	El sistema debe permitir a los revisores listar incidencias con paginación y filtrar por estado.	Alta
RF-06	El sistema debe permitir a los revisores cambiar el estado de una incidencia (Pendiente → En Proceso → Resuelto) agregando un comentario.	Alta
RF-07	El sistema debe permitir a los revisores subir una fotografía asociada a una incidencia (formato jpg/png, máximo 5MB).	Media
RF-08	El sistema debe registrar automáticamente en un historial cada cambio de estado con usuario, fecha y comentario.	Alta
RF-09	El sistema debe permitir a los administradores ver el historial de cambios de cualquier incidencia.	Media
RF-10	El sistema debe permitir a los administradores listar, crear, editar y eliminar edificios.	Alta

RF-11	El sistema debe permitir a los administradores listar, crear, editar y eliminar salones, validando que no se elimine un salón con incidencias asociadas.	Alta
RF-12	El sistema debe permitir a los administradores listar todos los usuarios registrados con su rol.	Media
RF-13	El sistema debe permitir a los administradores visualizar estadísticas de incidencias agrupadas por edificio y estado.	Alta
RF-14	El sistema debe permitir a los administradores exportar la lista de incidencias a un archivo CSV, con opción de filtrar por estado.	Baja
RF-15	El sistema debe implementar paginación en las listas de incidencias (5 o 10 por página) y salones (10 por página).	Alta

Cuadro 6.1: Requerimientos Funcionales

6.2. Requerimientos No Funcionales

ID	Descripción	Atributo
RNF-01	El tiempo de respuesta para la operación de login no debe superar los 2 segundos.	Rendimiento
RNF-02	La aplicación debe ser responsiva y funcionar correctamente en navegadores Chrome, Firefox y Edge (últimas versiones).	Usabilidad
RNF-03	El código fuente debe estar versionado en un repositorio Git con commits semánticos.	Mantenibilidad
RNF-04	La aplicación debe contenerizar la base de datos y el backend usando Docker Compose.	Portabilidad
RNF-05	Las contraseñas (si se llegaran a implementar) deben almacenarse hasheadas con bcrypt. Actualmente no se usan contraseñas.	Seguridad
RNF-06	El sistema debe manejar errores de conexión a base de datos y retornar mensajes claros al cliente.	Confiabilidad
RNF-07	El frontend debe aplicar estilos modernos (sombras, bordes redondeados, transiciones) para mejorar la experiencia de usuario.	Usabilidad

Cuadro 6.2: Requerimientos No Funcionales

6.3. Reglas de Negocio

RN-01	Una incidencia no puede cambiar de estado si ya se encuentra en Resuelto”.
RN-02	No se puede eliminar un salón que tenga incidencias asociadas.
RN-03	Un mismo nombre de salón puede existir en diferentes edificios (ej. A110 en A1 y A110 en A2) pero no en el mismo edificio.
RN-04	Los roles disponibles son únicamente: Admin, Revisor, Usuario.
RN-05	El comentario al cambiar de estado es opcional, pero recomendado.
RN-06	La subida de imágenes está limitada a 5MB y formatos JPG, JPEG, PNG.
RN-07	El token JWT tiene una duración de 8 horas; pasado ese tiempo el usuario debe volver a iniciar sesión.

Cuadro 6.3: Reglas de Negocio

Capítulo 7

Casos de Uso

Los casos de uso representan las interacciones entre los actores y el sistema. Los actores identificados son:

- **Usuario:** Persona de la comunidad universitaria que reporta incidencias.
- **Revisor:** Personal de limpieza que atiende y resuelve incidencias.
- **Administrador:** Coordinador que gestiona catálogos y visualiza reportes.

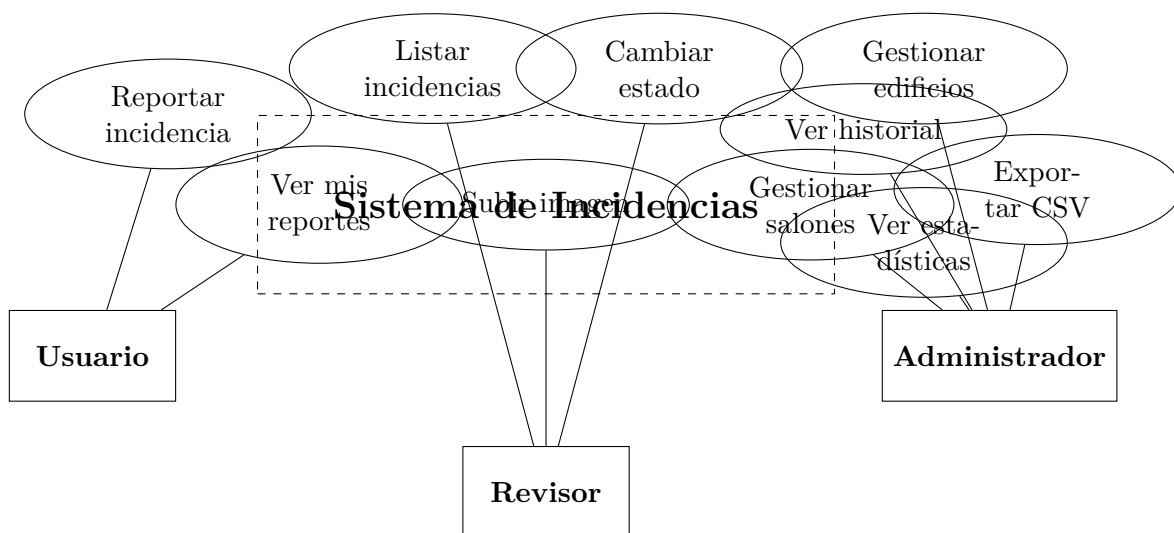


Figura 7.1: Diagrama de casos de uso del Sistema de Incidencias

La siguiente tabla detalla los casos de uso principales:

Caso de Uso	Actor	Descripción
-------------	-------	-------------

CU-01: Login	Usuario, Revisor, Admin	El actor ingresa correo, nombre y rol; el sistema verifica/registra al usuario y devuelve un JWT.
CU-02: Reportar incidencia	Usuario	El actor selecciona edificio, salón y escribe una descripción; el sistema almacena la incidencia con estado "Pendiente".
CU-03: Listar incidencias	Revisor, Admin	El actor visualiza una tabla paginada de incidencias con filtro por estado.
CU-04: Cambiar estado	Revisor	El actor selecciona una incidencia, elige nuevo estado (En Proceso/Resuelto) y opcionalmente agrega un comentario; el sistema actualiza el estado y registra el cambio en el historial.
CU-05: Subir imagen	Revisor	El actor adjunta una imagen a una incidencia; el sistema guarda la imagen en el servidor y actualiza la ruta en la base de datos.
CU-06: CRUD Edificios	Admin	El actor puede crear, leer, actualizar y eliminar edificios.
CU-07: CRUD Salones	Admin	El actor gestiona salones, con validación de no eliminación si tienen incidencias.
CU-08: Ver estadísticas	Admin	El actor visualiza un reporte agrupado por edificio y estado.
CU-09: Exportar CSV	Admin	El actor exporta la lista de incidencias (filtrable) a un archivo CSV.
CU-10: Ver historial	Admin	El actor consulta el historial de cambios de una incidencia específica.

Cuadro 7.1: Descripción de casos de uso

Capítulo 8

Historias de Usuario

Rol	Historia	Criterios de aceptación
Usuario	Como usuario, quiero reportar un problema de limpieza seleccionando el edificio y salón, para que el personal sepa exactamente dónde atender.	- El formulario permite elegir edificio y salón de manera dinámica. - Se envía correctamente la incidencia con estado Pendiente.
Revisor	Como revisor, quiero filtrar las incidencias por estado (Pendiente, En Proceso) para priorizar mi trabajo.	- Hay botones de filtro que recargan la tabla. - Solo se muestran las incidencias del estado seleccionado.
Revisor	Como revisor, quiero cambiar el estado de una incidencia a . ^{En Proceso.} Resuelto. ^{agregando un comentario,} para que quede evidencia de mi acción.	- Al hacer clic en . ^{En Proceso.} Resolver”, aparece un prompt para comentario. - El estado se actualiza y se registra en el historial.
Revisor	Como revisor, quiero subir una foto de la incidencia resuelta para demostrar que quedó limpia.	- Existe un botón ”Subir foto”que abre el selector de archivos. - La imagen se guarda y se muestra un enlace para verla.
Administrador	Como administrador, quiero gestionar edificios (agregar, editar, eliminar) para mantener actualizado el catálogo.	- La interfaz muestra la lista de edificios con botones Editar/Eliminar. - Al eliminar, se solicita confirmación.
Administrador	Como administrador, quiero ver un historial de cambios de cada incidencia para auditar el proceso.	- Cada incidencia tiene un botón ”Ver historial”que muestra en un modal fechas, estados anteriores/-nuevos, usuario y comentario.

Adminis- trador	Como administrador, quiero exportar todas las incidencias a CSV para analizarlas en Excel.	- Al hacer clic en ". ^E xportar CSV", se descarga un archivo con los datos de incidencias (filtro aplicado).
--------------------	--	---

Cuadro 8.1: Historias de usuario

Capítulo 9

Tecnologías Seleccionadas

9.1. Backend

- **Node.js:** Entorno de ejecución JavaScript asíncrono, ideal para aplicaciones I/O intensivas como APIs REST. Su ecosistema npm ofrece una amplia gama de módulos.
- **Express.js:** Framework minimalista que simplifica el enrutamiento, middlewares y manejo de peticiones. Se eligió por su flexibilidad y baja curva de aprendizaje.
- **JSON Web Tokens (JWT):** Mecanismo stateless de autenticación. El token se genera al login y se envía en cada petición vía header Authorization. Su expiración de 8 horas equilibra seguridad y usabilidad.
- **Multer:** Middleware para manejo de multipart/form-data, utilizado para subir imágenes. Se configuró con límite de 5MB y almacenamiento en disco.
- **json2csv:** Librería para convertir datos JSON a formato CSV, empleada en la exportación de reportes.

9.2. Frontend

- **HTML5, CSS3, JavaScript vanilla:** Se optó por tecnologías nativas sin frameworks para mantener la simplicidad, reducir dependencias y garantizar rendimiento. El CSS utiliza variables, flexbox, grid y animaciones modernas.
- **Fetch API:** Para realizar peticiones asíncronas al backend, con manejo de tokens JWT en los headers.
- **Diseño responsivo:** Se implementaron consultas de medios y unidades relativas para adaptarse a dispositivos móviles.

9.3. Base de Datos

- **MariaDB 10.11:** Sistema relacional elegido por su compatibilidad con MySQL, rendimiento y comunidad activa. Se emplean transacciones implícitas y claves foráneas con `ON DELETE CASCADE`.
- **Pool de conexiones:** La librería `mariadb` permite crear un pool de conexiones (límite de 5) para reutilizar conexiones y mejorar la concurrencia.

9.4. Contenedores y Despliegue

- **Docker:** Permite empaquetar la aplicación y sus dependencias en imágenes independientes del sistema operativo.
- **Docker Compose:** Orquesta dos servicios: `db` (MariaDB) y `app` (Node.js). Define redes y volúmenes para persistencia.
- **Script `init.sql`:** Se monta en el directorio de inicialización de MariaDB para crear tablas y cargar datos por defecto.

9.5. Control de Versiones

- **Git:** Repositorio local y remoto (GitHub). Se utiliza flujo GitFlow: ramas `main` (producción), `develop` (integración) y `feature/*` (nuevas funcionalidades).

Capítulo 10

Arquitectura del Sistema

El sistema sigue una arquitectura cliente-servidor de tres capas (presentación, lógica de negocio y datos).

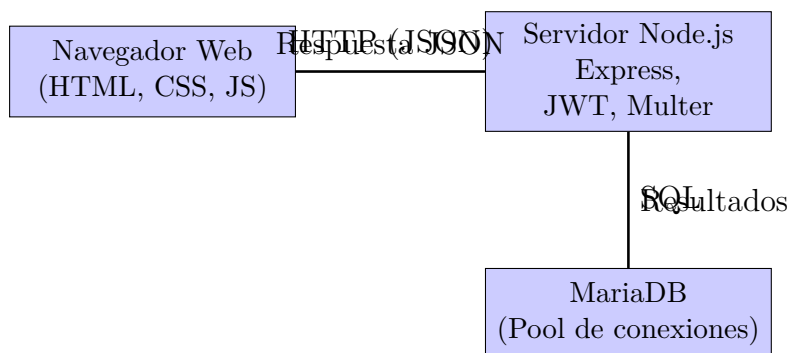


Figura 10.1: Arquitectura general cliente-servidor

10.1. Capa de Presentación

Las vistas HTML (`login.html`, `usuario.html`, `revisor.html`, `admin.html`) se sirven estáticamente desde `src/public/views`. El archivo `main.js` contiene toda la lógica de interacción: manejo de eventos, llamadas a la API mediante `fetchWithAuth`, actualización dinámica del DOM y gestión de paginación. El CSS (`styles.css`) define un sistema de diseño basado en tarjetas, pestañas y tablas responsivas.

10.2. Capa de Lógica de Negocio

El backend está construido con Express.js. Los archivos clave son:

- `server.js`: Configura middlewares, sirve archivos estáticos y monta las rutas bajo `/api`.
- `routes/authRoutes.js`: Define POST `/api/login` que llama al controlador de autenticación.
- `routes/incidenciaRoutes.js`: Define todas las rutas protegidas (incidencias, edificios, salones, estadísticas, exportación) aplicando los middlewares `verificarToken` y `verificarRol`.
- `controllers/authController.js`: Maneja el login/registro automático y la generación de JWT.
- `controllers/incidenciaController.js`: Contiene la lógica para cada endpoint: paginación, cambio de estado con historial, subida de imágenes, CRUD, etc.
- `config/jwt.js`: Funciones para generar token, verificar token y verificar roles.
- `config/db.js`: Configuración del pool de conexiones a MariaDB.

10.3. Capa de Datos

El modelo (`models/incidenciaModel.js`) agrupa todas las consultas SQL parametrizadas. Utiliza el pool de conexiones para obtener conexiones, ejecutar queries y liberarlas en un bloque `finally`. Las principales funciones son:

- `getIncidenciasPaginadas`: Devuelve un objeto con `data`, `total`, `pagina`, `totalPaginas`.
- `updateEstadoIncidenciaConHistorial`: Actualiza el estado y llama a `registrarHistorial`.
- `getSalonesPaginados`, `createEdificio`, etc.

10.4. Seguridad

- Autenticación JWT: Cada petición a rutas protegidas debe incluir el token en el header `Authorization: Bearer <token>`.
- Middleware `verificarRol`: Se aplica en rutas administrativas para restringir acceso solo a usuarios con rol `'Admin'`.
- Validación de entrada: Los controladores verifican campos obligatorios y lanzan errores apropiados.
- Prevención de inyección SQL: Todas las consultas usan parámetros con `?`.

Capítulo 11

Estructura del Proyecto y Distribución

El árbol de directorios del proyecto es el siguiente:

```
ProyectoVinculacion/  
  .env  
  .gitignore  
  docker-compose.yml  
  Dockerfile  
  init.sql  
  package.json  
  server.js  
  src/  
    config/  
      db.js  
      jwt.js  
    controllers/  
      authController.js  
      incidenciaController.js  
    models/  
      incidenciaModel.js  
    routes/  
      authRoutes.js  
      incidenciaRoutes.js  
    public/  
      css/  
        styles.css  
      js/  
        main.js  
      uploads/      (carpeta generada para imágenes)  
    views/
```

```
admin.html
login.html
revisor.html
usuario.html
README.md
```

11.1. Rol de los módulos

- `config/`: Configuración de base de datos y JWT.
- `controllers/`: Lógica de respuesta a las peticiones.
- `models/`: Acceso a datos (consultas SQL).
- `routes/`: Definición de endpoints y middlewares.
- `public/`: Archivos estáticos (CSS, JS, imágenes subidas).
- `views/`: Archivos HTML (interfaces de usuario).
- `init.sql`: Script de inicialización de la base de datos.
- Archivos raíz: configuración de Docker, dependencias y punto de entrada.

Capítulo 12

Uso de Contenedores y Despliegue

La aplicación está completamente contenerizada con Docker. El archivo `docker-compose.yml` define dos servicios:

- **db**: Usa la imagen `mariadb:10.11`. Se le asignan variables de entorno para la contraseña root, creación de base de datos y usuario. Mapea el puerto 3307 del host al 3306 interno. Monta un volumen `db_data` para persistencia y el archivo `init.sql` en `/docker-entrypoint-initdb.d/` para que se ejecute al iniciar.
- **app**: Construye la imagen a partir del `Dockerfile` en el contexto actual. Expone el puerto 3000 y depende del servicio `db`. Lee las variables de entorno del archivo `.env`.

El `Dockerfile` utiliza `node:18-alpine` como base, copia los archivos, instala dependencias con `npm install` y ejecuta `node server.js`.

Para desplegar el sistema, se ejecuta:

```
1 docker-compose up -d --build
```

Listing 12.1: Comandos de despliegue

Esto construye las imágenes, levanta los contenedores y la aplicación queda accesible en `http://localhost:3000`. El volumen `db_data` garantiza que los datos no se pierdan al reiniciar los contenedores.

La red interna `incidencias_net` permite la comunicación entre `app` y `db` usando el nombre del servicio como hostname (por eso `DB_HOST=db` en el `.env`).

Capítulo 13

Modelo de Base de Datos

El esquema relacional consta de seis tablas: `roles`, `usuarios`, `edificios`, `salones`, `incidencias` e `incidencias_historial`.

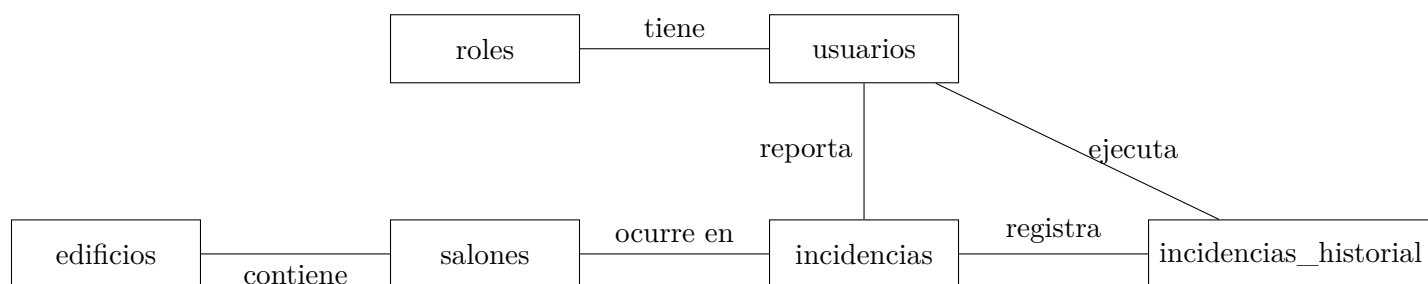


Figura 13.1: Modelo entidad-relación simplificado

13.1. Tablas y relaciones

Tabla	Llave primaria	Descripción y campos relevantes
roles	id	Almacena los roles: Admin (1), Revisor (2), Usuario (3).
usuarios	id	Contiene correo (único), nombre, rol_id (FK a roles).
edificios	id	Nombre del edificio (ej. A1, Lab2).
salones	id	Nombre del salón (ej. A110) y edificio_id (FK a edificios). Restricción UNIQUE (nombre, edificio_id).
incidencias	id	Descripción, usuario_id (FK a usuarios), salon_id (FK a salones), estado (ENUM: Pendiente, En Proceso, Resuelto), comentario, imagen_path, fechas.

incidencias_his- torial	id	incidencia_id (FK a incidencias), esta- do_anterior, estado_nuevo, comentario, usuario_id (FK a usuarios), fecha_cambio.
----------------------------	----	--

Cuadro 13.1: Descripción de tablas

13.2. Índices

Para optimizar las consultas más frecuentes se crearon índices:

```

1 CREATE INDEX idx_incidencias_estado ON incidencias(estado);
2 CREATE INDEX idx_incidencias_salon ON incidencias(salon_id);
3 CREATE INDEX idx_salones_edificio ON salones(edificio_id);
4 CREATE INDEX idx_historial_incidencia ON incidencias_historial(
    incidencia_id);

```

Listing 13.1: Índices creados

13.3. Integridad referencial

Todas las claves foráneas usan **ON DELETE CASCADE** excepto en **salones** donde se implementa una validación adicional en el modelo para evitar eliminar salones con incidencias activas.

Capítulo 14

Estrategia de Control de Versiones

El repositorio se aloja en GitHub (<https://github.com/ArielHernandez17/ProyectoVinculacion>). Se sigue el flujo GitFlow adaptado para un solo desarrollador:

- **Rama main:** Contiene el código estable y desplegable. Cada versión etiquetada con un tag semántico (v1.0.0).
- **Rama develop:** Rama de integración donde se fusionan las funcionalidades completadas.
- **Ramas feature/*:** Se crean a partir de **develop** para nuevas características (ej. `feature/paginacion`, `feature/historial`). Al finalizar, se fusionan mediante pull request.
- **Commits semánticos:** Se utilizan prefijos como `feat:`, `fix:`, `docs:`, `style:`, `refactor:` para facilitar la generación de changelogs.

Ejemplo de flujo de trabajo:

```
1 git checkout -b feature/paginacion develop
2 # Realizar cambios, commits
3 git add . && git commit -m "feat:␣agregar␣paginacion␣a␣incidencias"
4 git push origin feature/paginacion
5 # Crear Pull Request en GitHub, revisar y fusionar a develop
6 git checkout develop && git pull
7 git checkout main && git merge develop --no-ff
8 git tag -a v1.0.0 -m "Primera␣version␣estable"
9 git push --tags
```

Listing 14.1: Flujo Git

Capítulo 15

Riesgos del Proyecto

Se identificaron los siguientes riesgos técnicos y organizacionales:

Riesgo	Probabilidad	Impacto	Mitigación
Falla en la conexión a base de datos	Media	Alto	Reintentos con backoff exponencial en el pool de conexiones y logs detallados.
Pérdida de datos por caída del contenedor	Baja	Alto	Volúmenes Docker (<code>db_data</code>) y respaldos periódicos.
Vulnerabilidades de seguridad en JWT	Baja	Alto	Secretos largos en variables de entorno y expiración de 8 horas.
Sobrecarga de almacenamiento por imágenes	Media	Medio	Límite de 5MB por imagen; plan de limpieza de archivos antiguos.
Incompatibilidad del navegador con CSS moderno	Baja	Bajo	Autoprefixer y pruebas en navegadores objetivo.
Falta de adopción por parte de los usuarios	Media	Medio	Capacitación breve, documentación interactiva y retroalimentación continua.

Cuadro 15.1: Matriz de riesgos

Capítulo 16

Conclusiones

El desarrollo del Sistema de Incidencias ha permitido materializar una solución tecnológica que aborda de manera efectiva el problema de la gestión manual de reportes de limpieza en la FES Aragón.

16.1. Beneficios del sistema

- **Digitalización completa:** Se eliminó el uso de formatos físicos, centralizando la información en una base de datos accesible desde cualquier navegador.
- **Trazabilidad total:** Cada cambio de estado queda registrado en el historial, con usuario, fecha y comentario, permitiendo auditorías.
- **Reducción de tiempos:** Los revisores pueden priorizar incidencias pendientes y actualizar estados en tiempo real, reduciendo el tiempo de respuesta.
- **Indicadores confiables:** Las estadísticas por edificio y estado ayudan a la administración a detectar patrones y asignar recursos.
- **Portabilidad:** Gracias a Docker, el sistema se puede desplegar en cualquier servidor con un solo comando.

16.2. Aprendizajes del desarrollo

- Dominio de la arquitectura MVC en Node.js, separación de responsabilidades en controladores, modelos y rutas.
- Implementación de autenticación JWT y control de acceso basado en roles.
- Manejo de operaciones asíncronas con `async/await` y gestión de conexiones a base de datos mediante pool.

- Integración de subida de archivos con Multer y almacenamiento en disco.
- Contenerización completa con Docker Compose, incluyendo inicialización automática de la base de datos.
- Creación de interfaces de usuario dinámicas con JavaScript vanilla (sin frameworks), paginación, modales y notificaciones.

El proyecto cumple todos los objetivos planteados y deja abierta la posibilidad de futuras mejoras, como notificaciones por correo, integración con autenticación institucional (LDAP) o un panel de control en tiempo real.

Bibliografía

- [1] Node.js Foundation. (2023). *Node.js Documentation*. Recuperado de <https://nodejs.org/en/docs/>
- [2] Express.js Team. (2023). *Express - Guía de referencia de la API*. Recuperado de <https://expressjs.com/es/>
- [3] MariaDB Corporation. (2023). *MariaDB Server Documentation*. Recuperado de <https://mariadb.com/kb/en/documentation/>
- [4] Docker Inc. (2023). *Docker Documentation*. Recuperado de <https://docs.docker.com/>
- [5] Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)*. RFC 7519. IETF.

Matriz de Trazabilidad

Objetivo específico	Requerimientos funcionales asociados	Casos de uso
1. Diseño de BD	RF-04, RF-05, RF-06, RF-08, RF-10, RF-11	CU-02, CU-04, CU-06, CU-07
2. Backend con Node.js y API REST	RF-01, RF-02, RF-03, RF-05, RF-06, RF-07, RF-08, RF-09, RF-10, RF-11, RF-12, RF-13, RF-14, RF-15	Todos
3. Frontend responsivo	RF-03, RF-04, RF-05, RF-06, RF-07, RF-09, RF-10, RF-11, RF-12, RF-13, RF-14	Todos
4. Historial de cambios	RF-08, RF-09	CU-04, CU-10
5. Containerización con Docker	RNF-04	Ninguno

Cuadro 16.1: Matriz de trazabilidad objetivos vs requerimientos